Homework 3
Indicizzazione e ricerca di file di testo con Elasticsearch

Antonio Ciuffreda

13 novembre 2026

Repository GitHub: `https://github.com/ant-ciuffreda/homework-3`

1 Introduzione

Il progetto indicizza una cartella di file `.txt` con Elasticsearch, usando due indici (nome del file e contenuto) con analyzer diversi, e permette di interrogarli da console con una sintassi semplice. Il codice è scritto in Python e comunica con Elasticsearch attraverso il pacchetto ufficiale `elasticsearch` (versione [**9.5.1**], Python [**3.13.14**]). Elasticsearch (versione 9.5.6) è eseguito in un container Docker, su un computer Windows. Le istruzioni per eseguire il progetto sono nel file `README.md` della repository.

2 Dati

Abbiamo usato 45 articoli del dataset BBC News (Kaggle): 25 della categoria *business* e 20 della categoria *tech*. I file si chiamavano da `001.txt` a `045.txt`. Poiché la ricerca va fatta anche sul nome, con questi nomi l'indice dei nomi non avrebbe avuto nulla da analizzare. Abbiamo quindi scritto lo script `rinomina.py`, che costruisce un nuovo nome con la categoria, il numero e le prime parole del titolo dell'articolo, per esempio `business_001_ad_sales_boost_time_warner.txt`. I file sono stati rinominati prima di indicizzarli e il loro contenuto non è stato modificato.

3 Indici e analyzer

Un analyzer è il procedimento con cui Elasticsearch spezza un testo in parole (*token*) e le normalizza. L'abbiamo scelto per ciascun campo in base a come ci aspettiamo che gli utenti cerchino; lo stesso analyzer viene applicato anche alla query.

Indice	Campo	Analyzer
`homework3-nomi`	`nome`	personalizzato: tokenizer `whitespace` e filtri `word_delimiter_graph` e `lowercase`
`homework3-contenuti`	`contenuto`	`english` (predefinito)

In entrambi gli indici il campo `file` è di tipo `keyword`: resta intero e serve solo a mostrare il nome del file nei risultati.

3.1 Analyzer del nome

I nomi sono formati da parole separate da underscore. Con l'analyzer `standard` il nome resta un unico token, mentre con quello scelto viene spezzato in parole:

```
standard      -> ['business_001_ad_sales_boost_time_warner']
analyzer_nome -> ['business', '001', 'ad', 'sales', 'boost', 'time', 'warner']
```

Con `standard` una ricerca di `warner` non avrebbe trovato il file. Non abbiamo usato stemming né stopword, perché i nomi non sono testo scritto in frasi: ridurre "boost" o togliere "ad" non avrebbe senso.

3.2 Analyzer del contenuto

Gli articoli sono in inglese, quindi abbiamo usato l'analyzer `english`, che oltre alle minuscole toglie le parole molto comuni (*stopword*) e riduce le parole alla radice (*stemming*):

```
standard -> ['the', 'companies', 'are', 'growing', 'their', 'profits']
english  -> ['compani', 'grow', 'profit']
```

Grazie allo stemming, `company` e `companies` danno gli stessi risultati. Lo svantaggio è che una query formata solo da stopword (come `the`) non trova nulla, perché l'analyzer la elimina anche dalla query. Per controllare gli analyzer abbiamo usato lo script `verifica_analyzer.py`, che mostra i token prodotti, e abbiamo verificato che i campi del mapping usano davvero gli analyzer scelti (output in `risultati/verifica_analyzer.txt`).

4 Indicizzazione

Sono stati indicizzati **45 file**, e in entrambi gli indici ci sono 45 documenti. Ogni file è inviato a Elasticsearch con una richiesta separata e alla fine si forza un *refresh* per renderli cercabili. Abbiamo misurato il tempo con `time.time()` ed eseguito lo script tre volte (computer: [**processore e RAM**]).

Esecuzione	Indice nomi (s)	Indice contenuti (s)
1	0,380	0,425
2	0,352	0,415
3	0,372	0,395
Media	0,368	0,412

I due tempi sono simili anche se i nomi sono molto più corti dei testi: con soli 45 documenti il tempo dipende soprattutto dal numero di richieste al server, non dalla lunghezza del testo. Per molti più file converrebbe l'indicizzazione in blocco (`_bulk`).

5 Query di test

Le query si scrivono con la sintassi `nome <termini>`, `contenuto <termini>` oppure `contenuto "frase esatta"`. Per i termini senza virgolette si usa la query `match` (basta che ne compaia uno), per le frasi `match_phrase`. Abbiamo eseguito 11 query con `test_query.py` (risultati in `risultati/test_query.txt`). Il risultato atteso viene da una ricerca di testo in VS Code, che non usa analyzer.

Nella query 11 i risultati hanno punteggi diversi (0,622 per i nomi di 6 parole e 0,585 per quelli di 7), anche se contengono tutti `business`: il ranking predefinito di Lucene, BM25, assegna un punteggio più alto ai campi più corti a parità di parole trovate.

N.	Query	Ricerca VS	Elasticsearch	Commento
1	`contenuto market`	19	19	Termine singolo
2	`contenuto MARKET`	19	19	Stessi file e punteggi: le maiuscole non contano
3	`contenuto companies`	12	20	Lo stemming riduce `companies` e `company` alla stessa radice (`compani`): i file trovati sono quelli con una delle due forme
4	`contenuto company`	13	20	Come la query 3: stessi file e stessi punteggi
5	`contenuto mobile phones`	10	10	Basta uno dei due termini
6	`contenuto "mobile phones"`	7	7	Frase esatta: meno risultati della 5
7	`contenuto "phones mobile"`	0	0	L'ordine delle parole conta nella frase
8	`contenuto the`	45	0	`the` è una stopword: viene eliminata anche dalla query, che resta vuota e non trova nulla
9	`contenuto xyzabc`	0	0	Parola inesistente
10	`nome warner`	1	1	Trova `business_001_ad_sales_boost_time_warner.txt`
11	`nome business`	25	25	Tutti i nomi della categoria *business*

6 Considerazioni e limiti

- Il corpus è piccolo (45 file): per questo i tempi sono molto bassi e il ranking si basa su poche differenze.
- I nomi dei file derivano dai titoli, quindi parte del testo compare sia nel nome sia nel contenuto.
- Con più termini senza virgolette si usa l'OR, che restituisce anche i file con uno solo dei termini; per richiedere tutti i termini servirebbe una query `bool`.
- Lo stemming dell'inglese riduce parole diverse alla stessa radice e può dare risultati non voluti.
- Gli indici sono ricreati a ogni esecuzione di `indicizza.py`, quindi non c'è aggiornamento incrementale.